MicroROM-NPU

Microinstruction Format & Field Definitions

Revision 0.2 | 48-bit vertical microinstruction | 2D routine-local addressing

Specification status

Field layout and symbolic encodings below supersede the earlier draft that used NEXT_ADDR[11:6] and omitted CALL_MUL. These definitions are an engineering draft pending shared-package implementation, legality tests, and RTL simulation.

1. Exact 48-bit field layout

The 48-bit control word is indexed as shown below. The 2D MicroROM address is {routine_id[3:0], micro_step[4:0]}; NEXT_ADDR is a 5-bit local step, not an absolute 6-bit ROM address.

Field	Width	Bits	Role
UOP	5	[47:43]	Selects datapath operation. NOP suppresses ordinary datapath write only; other independent controls may still act.
SRC1_SEL	6	[42:37]	Select first operand from architectural or internal state.
SRC2_SEL	6	[36:31]	Select second operand from architectural or internal state.
DST_SEL	6	[30:25]	Select destination for UOP result; x0 ignores writes.
NEGATE	1	[24]	Negate SRC2 only for supported operations; currently ADD.
IMM_SEL	3	[23:21]	Select decoded immediate or constant.
MEM_CTRL	3	[20:18]	Select no memory transaction, data access, or instruction read.
COND	3	[17:15]	Select registered flag condition for NEXT=COND.
NEXT	3	[14:12]	Select sequencer transition after step completion.
NEXT_ADDR	5	[11:7]	Local micro-step target within current routine.
CALL_MUL	1	[6]	Call internal repeated-add routine 9 when legal.
PC_CTRL	2	[5:4]	Control the one architectural PC.
FLAGS_CTRL	2	[3:2]	Preserve, update, or clear EQ/LT/LTU flags.
RESERVED	2	[1:0]	Must be 00.

Field order, MSB to LSB: UOP | SRC1_SEL | SRC2_SEL | DST_SEL | NEGATE | IMM_SEL | MEM_CTRL | COND | NEXT | NEXT_ADDR | CALL_MUL | PC_CTRL | FLAGS_CTRL | RESERVED

Capacity: 10 implemented routine IDs (0-9) × 32 local steps × 48 bits = 15,360 logical bits. Routine IDs 10-15 are invalid. Constant-based standard-cell synthesis may optimize unused/default entries; physical area must be measured after synthesis.

2. UOP - datapath operation

UOP	Name	Defined behavior / legality
00000	NOP	No ordinary datapath operation/write. Independent MEM_CTRL, PC_CTRL, NEXT, and other explicitly legal controls may still act.
00001	MOV	DST <- SRC1.
00010	ADD	DST <- SRC1 + SRC2, with 32-bit wraparound; NEGATE=1 optionally changes SRC2 to -SRC2.
00011	ADDI	DST <- SRC1 + selected immediate/constant, with 32-bit wraparound.
00100	NEG	DST <- -SRC1 in 32-bit two's-complement arithmetic. NEGATE must be 0.
00101	CMP	Compare SRC1 and SRC2. Legal with FLAGS_CTRL=01 to capture EQ, signed LT, and unsigned LTU.
00110	XOR_SIGN	DST <- sign(SRC1) XOR sign(SRC2), normalized to 0 or 1.
00111	CLR	DST <- 0.
01000	LOAD	Blocking data-memory read into DST: MEM_CTRL=001 for signed byte, or 010 for word. The memory controller presents a completed result once.
01001	STORE	Blocking word store from SRC1 to address MAR; requires MEM_CTRL=011.
01010	DECODE	Decode IR into instruction fields/immediates; set instruction_valid and decoded routine ID. No ordinary DST write.
01011	Reserved	PC_UPDATE removed; PC_CTRL owns architectural PC updates.
01100	Reserved	WRITE_RD removed; use DST_SEL=RD with a normal result-producing UOP.
01101	FETCH	Special control operation: MAR <- PC. Selector fields are unused and must be canonical zero.
01110	Reserved	UOP RETURN removed; NEXT=RETURN performs micro-subroutine return.
01111-11111	Reserved	Illegal microinstruction unless deliberately assigned in a future revision.

Selector rules

Selector codes are interpreted by the datapath selection logic. RS1/RS2/RD are dynamic selector codes derived from IR; they are not separate physical registers. Architectural and internal register banks remain distinct. x0 always reads zero and ignores writes.

3. Selector namespace

Selector value	Name	Meaning
0-31 (000000-011111)	x0-x31	Architectural 32-bit integer registers; selected value equals the register number. x0 is hardwired zero.
32 (100000)	A	Internal multiplication operand / activation magnitude.
33 (100001)	B	Internal multiplication operand / weight magnitude.
34 (100010)	Reserved	TMP was removed; no physical TMP register in current architecture.
35 (100011)	ACC	Internal signed 32-bit accumulator.
36 (100100)	PROD	Internal product/result temporary.
37 (100101)	CTR	Internal repeated-add loop counter.
38 (100110)	MDR	Memory data register.
39 (100111)	MAR	Memory address register.
40 (101000)	SIGN	Internal 0/1 product sign indicator.
41 (101001)	RS1	Dynamic source selector: architectural x[IR.rs1].
42 (101010)	RS2	Dynamic source selector: architectural x[IR.rs2].
43 (101011)	RD	Dynamic destination selector: architectural x[IR.rd].
44 (101100)	IR	Instruction register; read source and write destination where legal.
45 (101101)	PC	Architectural PC read selector; writes must use PC_CTRL, not DST_SEL.
46-63	Reserved	Do not infer behavior.

4. Immediate and memory controls

Field/code	Meaning
IMM_SEL 000	Constant 0.
IMM_SEL 001	Sign-extended constant -1.
IMM_SEL 010	Sign-extended I-type immediate from IR.
IMM_SEL 011	Sign-extended S-type immediate from IR.
IMM_SEL 100	Sign-extended B-type branch immediate from IR.
IMM_SEL 101	Sign-extended J-type jump immediate from IR.
IMM_SEL 110	Constant +1.
IMM_SEL 111	Reserved.
MEM_CTRL 000	No memory transaction.
MEM_CTRL 001	Read signed byte; sign-extend to 32 bits.
MEM_CTRL 010	Read 32-bit word.
MEM_CTRL 011	Write 32-bit word.
MEM_CTRL 100	Instruction-memory read. With UOP=NOP, the memory controller captures the returned instruction word into MDR.
MEM_CTRL 101-111	Reserved.

Access policy: LB may use any byte address. Initial LW/SW implementation assumes naturally 4-byte-aligned addresses; misaligned-access behavior must be specified (for this version, report an access fault rather than silently split the transfer).

5. Conditions and microsequencer transitions

COND	Condition	Interpretation
000	EQ	Registered equality flag.
001	NE	NOT EQ.
010	LT	Registered signed SRC1 < SRC2.
011	GE	NOT signed LT.
100	LTU	Registered unsigned SRC1 < SRC2.
101	GEU	NOT unsigned LTU.
110	ALWAYS	True; meaningful only with NEXT=COND.
111	Reserved	Illegal.

NEXT	Mode	Action after the current operation completes
000	SEQ	Advance local step by one; step 31 cannot sequentially fall through.
001	JUMP	Set local step to NEXT_ADDR in the current routine.
010	COND	If COND is true, jump to NEXT_ADDR in the current routine; otherwise advance local step.
011	DISPATCH	On a valid decoded instruction, set routine ID to decoded routine and step to 0. Invalid instruction enters the error state.
100	RETURN	Restore saved caller routine ID and local step from μRET; illegal if no return is outstanding.
101	RESTART	Set routine ID=0 and step=0; legal only when no multiplication call is outstanding.
110-111	Reserved	Illegal.

Registered comparison flags

FLAGS_CTRL=00 preserves EQ/LT/LTU; 01 updates all three by comparing SRC1 and SRC2 (legal with UOP=CMP); 10 clears all three flags; 11 is reserved. Branch microinstructions read the registered flags and do not update them.

6. Remaining fields and special transitions

Field	Code / rule
PC_CTRL=00	Hold architectural PC.
PC_CTRL=01	Advance PC by instruction length. Current implementation supports 32-bit instructions only, so increment is 4 bytes.
PC_CTRL=10	Load PC from the current ALU result. For branch/jump routines, compute MAR + B/J immediate in that micro-step; no TMP or IPC is required.
PC_CTRL=11	Reserved. At most one PC action may commit per clock cycle.
CALL_MUL=0	Use NEXT transition normally.
CALL_MUL=1	Legal only with NEXT=SEQ. Save caller routine ID and current step+1, then enter routine 9 step 0. One outstanding call maximum; nested calls unsupported; no call from routine 9.
μRET storage	Saved routine ID (4 bits), saved local step (5 bits), and valid bit. Call from step 31 is illegal because step+1 cannot be represented.
NEXT_ADDR	Canonical zero unless NEXT=JUMP or NEXT=COND. It is a local step address, 0-31.
RESERVED	Bits [1:0] must be 00.

7. Completion, stalls, and errors

A common step_complete signal gates every microsequencer transition. For ordinary single-cycle datapath operations it is asserted when the result and any requested state update can commit. For memory operations, it is asserted only when the complete transaction result is accepted/captured. During a pending request/response the sequencer holds the same routine and local step. The memory controller tracks request acceptance to avoid duplicate requests.

Invalid microinstruction policy

Decode and validate the complete 48-bit word before committing any architectural/internal write, flag update, memory request, or PC update. On any reserved code, illegal field combination, invalid target/return/call, or nonzero reserved bit, suppress side effects, latch illegal_microinstruction, and enter the stopped/error state. Invalid architectural instructions use a distinct illegal_instruction status.

8. Encoded microinstruction checksums (field pack examples)

Routine / step	Meaning	Packed word (48-bit)
R0 / 0	FETCH: MAR <- PC	48'h680000000000
R0 / 1	NOP + instruction read; blocking transaction	48'h000000100000
R0 / 2	MOV MDR -> IR	48'h0CC058000000
R0 / 3	NOP + PC advance	48'h000000000010
R0 / 4	DECODE	48'h500000000000
R0 / 5	NOP + DISPATCH	48'h000000003000
R1 / 0	ADDI RS1 + I-imm -> MAR	48'h1D204E400000
R1 / 1	LOAD signed byte -> MDR	48'h40004C040000
R1 / 2	MOV MDR -> RD; RESTART	48'h0CC056005000
R2 / 0	ADDI RS1 + I-imm -> MAR	48'h1D204E400000
R2 / 1	LOAD word -> MDR	48'h40004C080000
R2 / 2	MOV MDR -> RD; RESTART	48'h0CC056005000

The words above are packed according to this document’s bit positions and symbolic code tables. They are not yet RTL-simulation-verified; package constants and an independent pack/unpack unit test must verify each word before ROM freeze.